Change Closure Management Module (CCLM)

Change Closure Management Module (CCLM)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Change Management Standard (SCMS)

Category: Governance & Enforcement

Subcategory: Software Change Management Governance

Type: Core Module

Governed Space: Software Change Closure Governance

Module ID: SLGA-SCMS-CCLM-05

Origin ID: OOF-SLGA-SCMS-CCLM

Version: 1.0

Status: Canonical Open

Effective Date: 14.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: UCL — Universal Canonical Language (English)

Canonical Definition

Change Closure Management Module (CCLM) is the Core Module within Software Change Management Standard (SCMS) that governs how software change cases are evaluated, classified, resolved, and formally closed according to their intended outcomes, actual implementation results, verified software conditions, fulfilled lifecycle obligations, remaining uncertainties, and residual responsibilities.

CCLM establishes software change closure as an evidence-supported

governance determination concerning the disposition of an identifiable change case rather than a simple administrative status indicating that a modification has been completed.

The module distinguishes implementation completion from outcome

verification, successful change realization from unsuccessful or partial realization, administrative closure from verified resolution, and the closure of a change case from the elimination of all obligations associated with its consequences.

It requires the original change purpose, approved or otherwise legitimately

established scope, implemented modification, resulting software state, verification evidence, lifecycle completion records, deviations, unresolved conditions, and closure decision to remain connected.

CCLM recognizes that software changes may conclude through successful implementation, partial implementation, failed implementation, rejection, withdrawal, supersession, deferral, cancellation, or transfer into another governed change case.

It also recognizes that a change may produce its intended technical result while leaving security, compatibility, operational, recovery, or lifecycle obligations unresolved.

CCLM applies to human-directed changes, automated development and delivery

systems, AI-generated modifications, autonomous software agents, enterprise applications, cloud infrastructure, embedded software, and robotic systems.

Its principal governance output is the Software Change Closure Profile.

Operational Role

CCLM establishes the final governance disposition mechanism within SCMS.

It governs:

• Change closure identity

• Change case continuity

• Original purpose preservation

• Intended outcome identification

• Implemented modification reference

• Actual software state identification

• Intended–actual outcome comparison

• Implementation completion assessment

• Verification evidence assessment

• Lifecycle obligation reconciliation

• Coordination obligation reconciliation

• Change deviation assessment

• Unintended consequence recognition

• Partial implementation assessment

• Failed change disposition

• Rejected and withdrawn change disposition

• Supersession and transfer

• Residual obligation management

• Closure eligibility

• Closure decision

• Closure evidence

• Post-closure discovery

• Closure reopening

• Historical continuity

• Final change governance classification

CCLM does not replace technical verification, release authorization, deployment monitoring, incident resolution, or specialized lifecycle governance.

Operational Question

How can a software change be legitimately and verifiably closed when its intended modification, actual implementation, resulting operational state, lifecycle obligations, unresolved consequences, and responsible actors may differ, without confusing administrative completion with successful change resolution?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS Core Module Flow

Change Request Management Module (CRMM) → Change Impact Assessment Module

(CIAM) → Change Coordination Control Module (CHCM) → Lifecycle Re-entry Control Module (LRECM) → Change Closure Management Module (CCLM)

CCLM is the fifth Core Module within SCMS.

CRMM establishes the change request and its original purpose.

CIAM establishes the assessed consequences and verification implications.

CHCM establishes coordination conditions and related obligations.

LRECM identifies and tracks required lifecycle governance activities.

CCLM evaluates the resulting change case against these references and

establishes its legitimate disposition.

Closure may require additional assessment, verification, correction, or transfer when the evidence does not support the intended outcome.

• Governed Space

Software Change Closure Governance

This governed space concerns the final evaluation and disposition of

identifiable software change cases.

It begins when a change case requires a determination of whether its intended outcome has been achieved, its implementation has failed, its processing has been discontinued, or another legitimate disposition applies.

It includes:

Outcome Verification

Implementation Reconciliation

Lifecycle Obligation Reconciliation

Coordination Obligation Reconciliation

Deviation Assessment

Residual Obligation Identification

Closure Eligibility

Closure Decision

Historical Closure Continuity

Post-Closure Reassessment

The governed space continues where new evidence materially challenges an earlier closure conclusion.

Governed Object

The primary Governed Object is:

A Software Change Closure Case representing the evidence-supported

relationship between an identifiable software change, its original purpose, applicable decisions, implemented or unimplemented modification, actual software state, verification results, fulfilled and outstanding obligations, final disposition, and closure authority.

The governed object must preserve, where applicable:

Change Closure Case ID

Change Request ID

Change Impact Assessment Reference

Change Coordination Reference

Lifecycle Re-entry Reference

Original Change Purpose

Intended Outcome

Applicable Change Decision

Implementation Reference

Actual Software State

Verification Evidence

Deviation Record

Residual Obligations

Closure Classification

Closure Decision

Closure Authority

Closure Time

Closure Evidence

Closure Version

Scope

CCLM governs:

Closure Case Formation

Change Purpose Reconciliation

Intended Outcome Assessment

Implementation Status Assessment

Actual State Identification

Outcome Verification

Change Success Classification

Partial Outcome Assessment

Failed Change Disposition

Rejected Change Disposition

Withdrawn Change Disposition

Superseded Change Disposition

Deferred Change Disposition

Lifecycle Obligation Reconciliation

Coordination Obligation Reconciliation

Residual Obligation Management

Closure Eligibility

Closure Authority

Closure Evidence

Post-Closure Reassessment

Closure Reopening

Historical Continuity

CCLM does not prescribe a specific issue-tracking system, software delivery methodology, incident management platform, or organizational closure workflow.

Foundational Principle

A software change may be considered successfully resolved only when its intended outcome is supported by applicable evidence, its resulting software state is adequately established, and its material governance obligations have been satisfied or explicitly preserved under a legitimate disposition.

Technical completion is not sufficient evidence of successful change resolution.

A change may be implemented without producing the expected outcome.

A defect may be corrected while introducing another defect.

A security vulnerability may be remediated while leaving operational compatibility unresolved.

A deployment may succeed while required monitoring remains incomplete.

An autonomous agent may report task completion without establishing that its actions remained within legitimate authority.

Implementation Complete ≠ Intended Outcome Verified.

Administrative Closure ≠ Successful Change Resolution.

Closure Case Identity

Every governed closure case must have a stable identity.

Relevant elements include:

Closure Case ID

Change Request ID

Software Product ID

Closure Origin

Responsible Closure Function

Closure Assessment Time

Applicable Change Version

Closure Status

Evidence Reference

Closure Version

A closure case must preserve its relationship to the original software change throughout revisions and reassessment.

Closure Trigger

Closure assessment may be triggered by:

Implementation Reported Complete

Verification Completed

Change Objective Achieved

Implementation Failed

Change Rejected

Change Withdrawn

Change Superseded

Change Deferred

Implementation Cancelled

Alternative Resolution Established

Change Transferred

Operational Conditions Require Final Disposition

A closure trigger does not establish closure eligibility.

Closure Purpose

The closure purpose must identify what the closure assessment is intended to establish.

Potential purposes include:

Confirm Successful Change Resolution

Determine Partial Completion

Classify Failed Implementation

Record Legitimate Rejection

Record Withdrawal

Establish Supersession

Document Deferral

Transfer Remaining Obligations

Preserve Final Evidence

Establish Closure Authority and Decision

The purpose must remain distinct from the outcome ultimately established.

Original Change Purpose

CCLM must preserve the purpose established through CRMM.

Relevant elements include:

Original Problem or Opportunity

Requested Modification

Intended Benefit

Affected Software Objects

Expected Result

Known Constraints

Original Evidence

Material Request Revisions

The original purpose must not be silently rewritten to match whatever result was achieved.

Intended Change Outcome

CCLM must identify the intended outcome against which the result is evaluated.

Relevant dimensions include:

Functional Outcome

Security Outcome

Quality Outcome

Compatibility Outcome

Dependency Outcome

Configuration Outcome

Operational Outcome

Performance Outcome

Data Integrity Outcome

Recovery Outcome

The intended outcome must remain connected to applicable change decisions and requirements.

Applicable Change Decision

CCLM must identify the decision under which the change proceeded or was otherwise disposed of.

Relevant elements include:

Decision Reference

Decision Authority

Approved or Accepted Scope

Applicable Conditions

Restrictions

Required Verification

Required Lifecycle Activities

Decision Validity

Decision Revisions

The existence of a decision does not establish that its conditions were satisfied.

Implementation Reference

Where implementation occurred, CCLM must identify the actual modification.

Relevant elements include:

Implemented Software Artifact

Source Modification

Dependency Change

Configuration Change

Data Migration

Firmware Modification

Implementation Actor

Execution Environment

Implementation Time

Execution Evidence

The implementation reference must remain distinguishable from the original proposal.

Implementation Identity

CCLM must establish which software state or artifact was actually modified.

Relevant elements include:

Software Product ID

Artifact ID

Version ID

Build Reference

Configuration Baseline

Dependency State

Deployment Reference

Affected Environment

Observed State

A reported implementation must not be accepted as sufficient when the resulting software object cannot be reliably identified.

Implementation Completion

Implementation completion concerns whether the intended technical

activities were performed.

Relevant elements include:

Required Modification Performed

Artifact Produced

Configuration Applied

Dependency Updated

Migration Executed

Implementation Evidence Available

Execution Errors

Uncompleted Activities

Implementation completion must remain separate from outcome verification and lifecycle governance completion.

Actual Software State

CCLM must identify the resulting software state relevant to closure.

Potential elements include:

Actual Software Version

Actual Artifact Identity

Actual Configuration

Actual Dependencies

Observed Functionality

Observed Security Conditions

Observed Quality Conditions

Observed Data State

Observed Operational Behavior

Known Restrictions

The actual state must be established through applicable evidence rather than inferred solely from implementation records.

Intended–Actual State Comparison

CCLM must compare the intended change outcome with the actual resulting state.

Relevant comparisons include:

Expected Functionality ↔ Observed Functionality

Target Artifact ↔ Actual Artifact

Target Configuration ↔ Actual Configuration

Expected Dependency State ↔ Actual Dependency State

Expected Security Condition ↔ Verified Security Condition

Expected Performance ↔ Observed Performance

Expected Operational Behavior ↔ Observed Operational Behavior

Material differences must remain identifiable.

Outcome Verification

Outcome verification establishes whether the intended result has been

achieved under applicable conditions.

Relevant elements include:

Verification Objective

Applicable Acceptance Criteria

Verification Method

Verification Environment

Evidence Reference

Observed Result

Known Limitations

Verification Conclusion

CCLM references verification performed under the appropriate lifecycle

mechanisms.

It does not replace their specialized technical procedures.

Outcome Evidence

CCLM must preserve evidence supporting the final outcome classification.

Potential evidence includes:

Test Results

Security Verification

Build and Artifact Records

Configuration Records

Dependency Compatibility Evidence

Release Records

Deployment Evidence

Operational Monitoring

Data Integrity Verification

Recovery Verification

Physical-System Validation

Responsible Function Assessment

Evidence must remain associated with the software state and conditions to which it applies.

Evidence Applicability

Closure evidence must be evaluated for applicability to the resulting software state.

Relevant elements include:

Artifact Identity

Software Version

Configuration

Dependency State

Verification Environment

Operational Conditions

Evidence Time

Verification Scope

Known Limitations

Evidence concerning an earlier software state must not automatically establish the acceptability of a materially modified state.

Verification Coverage

CCLM must evaluate whether the verification evidence covers the material closure conditions.

Relevant elements include:

Intended Outcome Coverage

Affected Software Object Coverage

Required Quality Verification

Required Security Verification

Compatibility Verification

Operational Verification

Recovery Verification

Material Residual Risk Assessment

Insufficient coverage must remain explicit.

Change Success Assessment

A change may be classified as successfully implemented when:

The Intended Modification Was Performed

The Applicable Outcome Criteria Were Satisfied

The Resulting Software State Was Identified

Required Verification Evidence Is Available

Material Lifecycle Obligations Are Satisfied

Applicable Authority Conditions Were Respected

No Unresolved Condition Prevents Successful Classification

Successful classification must be limited to the scope actually supported by evidence.

Partial Change Outcome

A change may achieve only part of its intended result.

Examples include:

Partial Functionality Delivered

One Defect Corrected While Another Remains

Security Exposure Reduced but Not Eliminated

Migration Partially Completed

Compatibility Established for Limited Environments

Operational Improvement Achieved With Restrictions

CCLM must preserve the achieved outcome and the unresolved portion separately.

Failed Change Outcome

A change may fail when:

The Intended Modification Cannot Be Completed

The Expected Outcome Is Not Achieved

Verification Fails

The Resulting State Is Unacceptable

Required Dependencies Cannot Be Satisfied

Operational Consequences Become Unacceptable

Implementation Must Be Reversed or Abandoned

Failure must remain a legitimate, evidence-supported disposition rather than being concealed through administrative closure.

Unintended Consequences

CCLM must identify material unintended consequences discovered during or after implementation.

Potential consequences include:

New Functional Defect

Security Exposure

Dependency Incompatibility

Configuration Conflict

Data Integrity Problem

Performance Degradation

Operational Instability

Recovery Limitation

Unexpected Agent Behavior

Physical-System Deviation

Unintended consequences may require impact reassessment, additional lifecycle processing, or a new change case.

Change Deviation

A change deviation occurs when actual implementation or resulting conditions differ materially from the established change scope or requirements.

Relevant elements include:

Expected Modification

Actual Modification

Deviation Origin

Affected Software Objects

Operational Consequences

Authority Implications

Required Reassessment

Disposition

A deviation must not be treated as acceptable merely because the resulting software appears functional.

Scope Reconciliation

CCLM must reconcile the scope originally established for the change with the actual implementation.

Relevant elements include:

Original Scope

Approved Scope

Implemented Scope

Unimplemented Scope

Additional Modifications

Affected Objects

Scope Deviations

Applicable Decisions

Material unapproved scope expansion must remain visible.

Impact Assessment Reconciliation

CCLM must compare the assessed consequences from CIAM with the observed outcome.

Relevant elements include:

Predicted Direct Impact

Observed Direct Impact

Predicted Indirect Impact

Observed Indirect Impact

Known Uncertainty

Newly Discovered Consequences

Impact Assumptions

Required Reassessment

Material differences may require the impact assessment to be reopened.

Coordination Obligation Reconciliation

CCLM must evaluate relevant obligations established through CHCM.

Potential elements include:

Required Coordination Activities

Participating Change Cases

Dependency Conditions

Compatibility Requirements

Conflict Resolution

Sequence Compliance

Coordination Evidence

Residual Coordination Obligations

Closure of one change must not conceal unresolved conflicts affecting related changes.

Lifecycle Obligation Reconciliation

CCLM must evaluate the obligations established through LRECM.

Relevant elements include:

Required Parent Standards

Required Core Modules

Required Activities

Entry Conditions

Verification Requirements

Evidence Reuse Decisions

Completed Activities

Failed Activities

Deferred Activities

Unresolved Obligations

A change cannot be classified as fully governance-complete when mandatory lifecycle obligations remain unsatisfied.

Requirements Closure

Where requirements were affected, CCLM must preserve relevant SRS conclusions.

Potential elements include:

Applicable Requirements

Requirement Changes

Requirements Traceability

Acceptance Evidence

Outstanding Requirements

Requirements Conflicts

CCLM does not independently replace requirements governance.

Creation and Artifact Closure

Where software creation activities were required, CCLM must preserve relevant SCS and SCFS conclusions.

Potential elements include:

Created Artifact

Creator Attribution

Implementation Traceability

Artifact Identity

Configuration Baseline

Build Evidence

Reproducibility Evidence

Unresolved Artifact Conditions

Technical artifact completion must remain distinguishable from verified change success.

Dependency Closure

Where dependencies were affected, CCLM must preserve relevant SDS conclusions.

Potential elements include:

Dependency Identity

Dependency Version

Suitability Evidence

Compatibility Evidence

Dependency Origin

Continuity Conditions

Residual Dependency Risk

An updated dependency does not automatically establish that every dependent software object remains compatible.

Quality Closure

Where quality governance was required, CCLM must preserve relevant SQS conclusions.

Potential elements include:

Required Tests

Executed Tests

Test Results

Regression Verification

Defect Status

Quality Condition Assessment

Unresolved Quality Findings

Passing a subset of tests must not substitute for satisfaction of the applicable quality requirements.

Security Closure

Where security governance was required, CCLM must preserve relevant SSS conclusions.

Potential elements include:

Security Requirements

Vulnerability Status

Security Testing

Remediation Evidence

Residual Exposure

Security Exceptions

Outstanding Security Obligations

A security change may be implemented while the underlying vulnerability remains unresolved.

Release Closure

Where release governance was required, CCLM must preserve relevant SRLS conclusions.

Potential elements include:

Release Candidate Identity

Release Readiness

Release Authorization

Release Conditions

Release Records

Release Restrictions

Withdrawal Requirements

Release readiness must not be inferred solely from the completion of development activities.

Deployment Closure

Where deployment governance was required, CCLM must preserve relevant SDPS conclusions.

Potential elements include:

Deployment Authorization

Production Version

Deployment State

Operational Monitoring

Deployment Verification

Rollback or Recovery

Unresolved Production Conditions

Successful deployment execution does not automatically establish successful change outcome.

Operational Outcome

CCLM must evaluate operational consequences where they are material to the change purpose.

Relevant elements include:

Service Availability

Functional Behavior

Performance

Resource Consumption

Security Conditions

User Impact

Data Integrity

System Compatibility

Recovery Capability

Operational Restrictions

Operational observations must be associated with the actual software state and relevant environment.

Post-Implementation Observation

Some outcomes cannot be established immediately after implementation.

Examples include:

Intermittent Defect Resolution

Long-Term Performance Improvement

Memory Leak Correction

Reliability Improvement

Security Monitoring Effectiveness

Resource Efficiency

Recovery Stability

Physical-System Behavior Over Repeated Operation

Where observation is necessary, CCLM must preserve the required observation conditions and completion criteria.

Observation Window

A required observation period must have identifiable conditions.

Relevant elements include:

Observation Purpose

Start Condition

Applicable Software State

Monitored Indicators

Required Duration or Event Coverage

Acceptance Criteria

Failure Conditions

Responsible Function

Evidence

An observation window must be justified by the governed change rather than treated as a universal fixed duration.

Outcome Uncertainty

CCLM must preserve uncertainty that materially affects closure conclusions.

Potential categories include:

Insufficient Verification

Unknown Operational Impact

Unverified Dependency Compatibility

Incomplete Monitoring

Unresolved Security Finding

Unknown Data Integrity Condition

Uncertain Recovery Capability

Conflicting Evidence

Unverified Physical-System Consequence

Uncertainty must not be converted into a successful outcome classification without an appropriate evidence basis.

Residual Obligation Identification

CCLM must identify obligations remaining after implementation or another change disposition.

Potential obligations include:

Additional Verification

Security Remediation

Compatibility Correction

Operational Monitoring

Data Migration Completion

Recovery Validation

Documentation Update

External Stakeholder Notification

Related Change Resolution

Exception Review

Each material obligation must remain identifiable.

Residual Obligation Ownership

Every material residual obligation must have an applicable responsibility or control reference.

Relevant elements include:

Obligation ID

Originating Change

Required Action

Responsible Function

Applicable Authority

Due Condition or Review Trigger

Evidence Requirement

Current Status

Escalation Condition

An obligation must not disappear merely because the originating change case is administratively closed.

Residual Obligation Transfer

An unresolved obligation may be transferred to another governed process where legitimate.

Potential destinations include:

New Software Change Case

Existing Related Change Case

Security Remediation Process

Defect Management

Deployment Recovery

Operational Monitoring

Migration Process

Retirement Governance

Transfer must preserve:

Original Obligation

Destination Reference

Receiving Responsibility

Transfer Decision

Evidence

Continuity

Transfer does not itself establish resolution.

Closure Eligibility

CCLM must determine whether the change case is eligible for its proposed closure classification.

Relevant elements include:

Identifiable Change Case

Applicable Disposition

Sufficient Supporting Evidence

Actual Outcome Established Where Relevant

Lifecycle Obligations Reconciled

Coordination Obligations Reconciled

Material Deviations Addressed

Residual Obligations Identified

Applicable Authority Established

Closure Conditions Satisfied

Eligibility depends on the proposed closure category.

Successful implementation and legitimate rejection require different

evidence.

Closure Classification

CCLM recognizes the following principal closure classifications:

Successfully Implemented and Verified

Implemented With Residual Obligations

Partially Implemented

Implementation Failed

Rejected Without Implementation

Withdrawn Without Implementation

Superseded

Cancelled

Deferred — Not Finally Resolved

Transferred to Another Governed Case

Closure Pending Verification

Closure Blocked

Reassessment Required

Outcome Unknown

Outcome Conflicted

A classification must not imply greater completion than the supporting evidence establishes.

Successful Closure

Successful closure requires evidence supporting the intended outcome and

applicable governance completion.

Relevant conditions include:

Intended Outcome Verified

Resulting Software State Identified

Required Lifecycle Activities Satisfied

Material Coordination Obligations Resolved

Applicable Authority Conditions Respected

No Unresolved Condition Invalidates Success

Closure Decision Recorded

Successful closure must remain limited to the verified change scope.

Closure With Residual Obligations

A change may have an established implementation outcome while retaining obligations that do not invalidate the specific closure classification.

Such closure requires:

Explicit Residual Obligation Identification

Applicable Responsibility

Transfer or Continuation Mechanism

Evidence Preservation

Restrictions Where Necessary

Follow-up Conditions

Competent Closure Decision

A case must not be described as fully resolved if material obligations remain unresolved.

Failed Change Closure

A failed change may be administratively closed under an explicit failed disposition.

Relevant conditions include:

Failure Identified

Actual Software State Established

Operational Consequences Assessed

Recovery or Restriction Requirements Identified

Residual Obligations Preserved

Required Follow-up Established

Closure Authority Recorded

Failed closure is not successful change resolution.

Rejected Change Closure

A rejected request may be closed without implementation.

CCLM must preserve:

Original Request

Rejection Decision

Applicable Authority

Rejection Reason

Supporting Evidence

Related Obligations

Final Request Status

Rejection of a proposed solution must not conceal an independently established defect, vulnerability, or operational obligation.

Withdrawn Change Closure

A withdrawn request may receive a final administrative disposition when the withdrawal is legitimate.

Relevant elements include:

Withdrawal Actor

Withdrawal Eligibility

Withdrawal Reason

Withdrawal Time

Implementation Status

Outstanding Obligations

Closure Decision

Withdrawal must not be treated as evidence that the original problem no longer exists.

Superseded Change Closure

A change may be superseded by another change case.

CCLM must preserve:

Original Change ID

Successor Change ID

Supersession Reason

Transferred Evidence

Transferred Obligations

Changed Scope

Applicable Decision

Continuity Reference

Supersession must not erase the history or unresolved consequences of the original change.

Deferred Change Disposition

A deferred change remains unresolved unless a separate legitimate final

disposition applies.

Relevant elements include:

Deferral Reason

Responsible Function

Pending Condition

Review Trigger

Operational Restriction

Residual Risk

Next Governance Action

Current Status

Deferral must not be represented as successful closure.

Closure Authority

CCLM must identify the competent authority or governance function responsible for the closure decision.

Relevant elements include:

Closure Decision Actor

Applicable Authority Basis

Decision Scope

Evidence Reviewed

Applicable Restrictions

Decision Conditions

Decision Time

Decision Record

Closure authority must remain distinct from implementation authority and technical verification responsibility.

Automated Closure

Automated systems may support closure assessment where legitimate criteria and authority conditions are established.

Relevant elements include:

Automation Identity

Applicable Change Case

Closure Criteria

Evidence Sources

Verification Conditions

Permitted Closure Categories

Exception Conditions

Escalation Requirements

Decision Evidence

Automated closure must not be based solely on pipeline completion or a work item status.

Autonomous Agent Closure Boundary

An autonomous agent may report completion, submit evidence, or recommend closure within its assigned scope.

CCLM must preserve:

Agent Identity

Assigned Objective

Permitted Actions

Implementation Evidence

Verification Evidence

Closure Recommendation

Applicable Decision Authority

Escalation Requirements

An agent's ability to execute a change does not automatically establish authority to declare that change successfully closed.

Agent Task Completion ≠ Authorized Change Closure.

Closure Evidence Integrity

CCLM must preserve the integrity and provenance of material closure evidence.

Relevant elements include:

Evidence ID

Evidence Source

Evidence Time

Associated Software State

Collection Method

Integrity Reference

Verification Status

Known Limitations

Decision Association

Closure conclusions must remain reconstructable from their supporting evidence.

Closure Record Continuity

The closure record must preserve the relationship between:

Original Change Request

Impact Assessment

Coordination Case

Lifecycle Re-entry Case

Implementation Record

Verification Evidence

Actual Software State

Residual Obligations

Closure Decision

Final Classification

Historical records must not be silently overwritten when closure is reassessed.

Post-Closure Discovery

New evidence may reveal that an earlier closure conclusion was incomplete or incorrect.

Potential triggers include:

Recurring Defect

New Security Finding

Operational Failure

Unexpected Dependency Consequence

Data Integrity Problem

Invalidated Verification Evidence

Unrecognized Implementation Deviation

Physical-System Incident

New evidence must be assessed for its relevance to the original closure basis.

Closure Reopening

A closure case may require reopening when material new evidence invalidates the previous disposition.

Reopening must preserve:

Original Closure Decision

Original Evidence

New Evidence

Reopening Reason

Affected Software State

Required Reassessment

Responsible Function

Revised Status

Historical Continuity

Reopening must not erase the earlier decision or its original basis.

Closure Finality

Closure finality concerns whether the case has reached a legitimate final

disposition under the applicable governance conditions.

Finality must distinguish:

Administrative Finality

Implementation Finality

Verification Finality

Lifecycle Obligation Finality

Operational Outcome Finality

These conditions may not occur simultaneously.

A final administrative record may remain historically valid as a record of the decision made at that time, even if later evidence requires a new assessment.

Final Record ≠ Immutable Truth About the Resulting Software State.

Closure Governance Classification

CCLM recognizes the following governance states:

Closure Assessment Required

Closure Assessment Active

Implementation Evidence Pending

Outcome Verification Pending

Lifecycle Reconciliation Pending

Coordination Reconciliation Pending

Residual Obligations Identified

Closure Eligible

Successfully Closed

Closed With Residual Obligations

Closed as Failed

Closed as Rejected

Closed as Withdrawn

Closed as Superseded

Deferred

Transferred

Closure Blocked

Reassessment Required

Reopened

Unknown

Conflicted

Each classification must preserve its evidence, scope, applicable decision, and time.

Governance Requirements

A conforming CCLM implementation must satisfy the following requirements. Closure Identity and Context

• Establish a stable closure case identity

• Associate closure with an identifiable change request

• Preserve the original change purpose

• Identify the intended outcome

• Preserve applicable change decisions

• Identify the implemented modification where applicable

• Establish the relevant actual software state

• Preserve closure assessment scope

Outcome and Evidence

• Compare intended and actual outcomes

• Identify applicable acceptance criteria

• Preserve outcome verification evidence

• Evaluate evidence applicability

• Evaluate verification coverage

• Identify material deviations

• Identify unintended consequences

• Preserve material uncertainty

• Distinguish implementation completion from outcome verification

Governance Reconciliation

• Preserve relevant impact assessment references

• Reconcile coordination obligations

• Reconcile lifecycle re-entry obligations

• Identify required specialized verification

• Identify incomplete governance activities

• Preserve applicable authority conditions

• Prevent incomplete governance from being classified as fully

satisfied.

Disposition and Obligations

• Establish a justified closure classification

• Distinguish successful, partial, failed, and non-implementation

• Identify residual obligations

• Assign or reference responsibility for residual obligations

• Preserve obligation transfer records

• Establish closure eligibility

• Identify competent closure authority

• Preserve the closure decision and its basis

• Prevent deferral from being represented as successful closure

• Prevent administrative closure from concealing unresolved material

dispositions. conditions.

Continuity and Reassessment

• Preserve closure evidence provenance

• Maintain the original change-to-closure traceability chain

• Identify post-closure reassessment triggers

• Support evidence-based reopening

• Preserve historical closure decisions

• Preserve revised closure classifications

• Maintain continuity of transferred obligations

• Distinguish final administrative disposition from continuing

operational reality.

Governance Logic

Change Request → Original Purpose → Applicable Change Decision → Impact

Assessment Reference → Coordination Reference → Lifecycle Re-entry Reference → Implementation Status → Actual Software State → Intended–Actual Comparison → Verification Evidence → Lifecycle Obligation Reconciliation → Coordination Obligation Reconciliation → Deviation Assessment → Residual Obligations → Closure Eligibility → Closure Authority → Closure Decision → Software Change Closure Profile

Where the intended outcome is verified:

Implemented Change → Applicable Verification → Obligations Satisfied →

Successful Closure

Where the outcome is partially achieved:

Implemented Change → Partial Outcome → Unresolved Requirements → Residual

Obligation Assessment → Qualified Disposition

Where implementation fails:

Implementation Failure → Actual State Assessment → Recovery or Restriction

Requirements → Failed Disposition → Follow-up Governance

Where the request is rejected or withdrawn:

Change Request → Legitimate Rejection / Withdrawal → Remaining Obligation

Assessment → Administrative Closure

Where the change is superseded:

Original Change → Successor Change → Evidence and Obligation Transfer →

Superseded Disposition

Where material evidence is insufficient:

Closure Request → Evidence Gap → Verification / Clarification /

Reassessment → Updated Closure Eligibility

Where new evidence emerges after closure:

Historical Closure → New Material Evidence → Closure Basis Reassessment →

Reopen / Preserve / Revise Disposition

CCLM must not equate the completion of a technical task, automated workflow, or administrative ticket with successful software change resolution.

Governance Outputs

The principal output is:

Software Change Closure Profile

Supporting outputs may include:

Change Closure Case Identity

Original Purpose Reference

Intended Outcome Profile

Applicable Change Decision Reference

Implementation Status Record

Actual Software State Profile

Intended–Actual Comparison

Outcome Verification Record

Verification Coverage Assessment

Impact Reconciliation Record

Coordination Obligation Reconciliation

Lifecycle Obligation Reconciliation

Deviation Register

Unintended Consequence Register

Residual Obligation Register

Residual Obligation Transfer Record

Closure Eligibility Assessment

Closure Classification

Closure Authority Reference

Closure Decision Record

Closure Evidence Register

Post-Closure Reassessment Record

Closure Reopening Record

Minimum Implementation Framework

A minimum CCLM implementation must establish eight governance layers. Layer 1 — Change Identity and Purpose

Identify the original change request, intended outcome, applicable

decisions, and relevant software objects. Layer 2 — Implementation and Actual State

Establish what was actually implemented, which software state resulted, and

which activities remain incomplete. Layer 3 — Outcome Verification

Compare the intended and actual results using applicable verification

evidence and acceptance criteria. Layer 4 — Governance Reconciliation

Evaluate the fulfillment of impact-related, coordination-related, and

lifecycle re-entry obligations. Layer 5 — Deviations and Uncertainty

Identify material differences, unintended consequences, evidence gaps, and

unresolved conditions. Layer 6 — Residual Obligation Control

Identify outstanding obligations, applicable responsibilities, transfer

conditions, and required follow-up. Layer 7 — Closure Decision

Establish closure eligibility, competent authority, final disposition, and

supporting evidence. Layer 8 — Continuity and Reassessment

Preserve the Software Change Closure Profile, historical evidence,

obligation continuity, and conditions for reopening.

The implementation may use manual governance records, issue-tracking systems, delivery platforms, automated verification, or AI-assisted assessment.

The methodological requirements remain independent of the technology used.

Cross-Module Boundary

CRMM → CCLM

CRMM establishes the original request, purpose, scope, and initial

disposition.

CCLM preserves those references and evaluates the final disposition of the

change case.

CIAM → CCLM

CIAM identifies expected consequences, uncertainty, and verification

implications.

CCLM compares relevant assessed consequences with the observed result and

identifies material differences.

CHCM → CCLM

CHCM establishes coordination requirements, conflicts, dependencies, and

residual coordination obligations.

CCLM reconciles their status as part of closure eligibility.

LRECM → CCLM

LRECM establishes required lifecycle activities, evidence, and outstanding

obligations.

CCLM evaluates whether those obligations have been satisfied, legitimately

excepted, or explicitly transferred.

CCLM does not replace the other SCMS modules. It establishes the final evidence-supported disposition of the change case using their governance outputs.

Cross-Architecture Boundary

CCLM evaluates software change closure using applicable SLGA™ governance

evidence.

SRS — Software Requirements Standard

Requirements satisfaction, change control, and traceability.

SCS — Software Creation Standard

Software creation, artifact attribution, and development traceability.

SDS — Software Dependency Standard

Dependency suitability, compatibility, and continuity.

SCFS — Software Configuration Standard

Configuration baselines, build evidence, artifact identity, and

reproducibility.

SQS — Software Quality Standard

Testing, defects, and quality condition assessment.

SSS — Software Security Standard

Security verification, vulnerabilities, and remediation.

SRLS — Software Release Standard

Release readiness, authorization, records, and withdrawal.

SDPS — Software Deployment Standard

Deployment state, production version, monitoring, rollback, and recovery.

SRTS — Software Retirement Standard

Migration, residual dependencies, and retirement closure.

SLCS — Software Lifecycle Control Standard

Lifecycle state consistency, transitions, exceptions, control transfer, and

continuity.

CCLM may also reference applicable OOF® authority, accountability, autonomous systems, and operational reality governance mechanisms where the resulting software change affects those domains.

Practical Applications

Enterprise Software Maintenance

Establishing evidence-supported closure of defects, enhancements,

configuration changes, and dependency updates.

Continuous Integration and Delivery

Preventing automated pipeline completion from being treated as proof of

successful software change resolution.

Security Remediation

Distinguishing completed security patches from verified remediation of the

underlying vulnerability.

AI-Assisted Software Engineering

Evaluating AI-generated modifications against their intended outcomes,

verification evidence, and applicable lifecycle obligations.

Autonomous Software Operations

Preventing autonomous agents from declaring successful closure merely

because assigned execution tasks have completed.

Critical Infrastructure

Preserving evidence of operational acceptability, unresolved restrictions,

and recovery obligations after consequential software changes.

Robotics and Embedded Systems

Establishing whether firmware and control logic modifications achieved

their intended results without creating unacceptable physical-system consequences.

• Use Case 1 — A Security Patch Is Deployed but the Vulnerability Remains

Exploitable

A software team receives a change request to correct a critical

authorization vulnerability.

A patch is developed, tested, and deployed.

The delivery pipeline reports successful completion.

The change ticket is automatically marked as resolved.

However, a subsequent security verification identifies another execution

path through which the original vulnerability remains exploitable.

CCLM evaluates:

Original Security Objective → Implemented Patch → Deployment Evidence →

Security Verification → Actual Security Condition → Closure Eligibility

The assessment identifies:

• The intended technical modification was implemented

• The deployment completed successfully

• The original security objective was not fully achieved

• The remaining vulnerability represents an unresolved material

• Successful closure is not supported by the available evidence

• Additional remediation and lifecycle reassessment are required

condition.

The appropriate classification is:

Implementation Completed — Security Outcome Not Verified — Closure

Blocked.

Successful Deployment ≠ Successful Security Remediation.

Use Case 2 — An Autonomous Agent Reports a Completed Fix While Leaving

a Data Integrity Obligation

An autonomous software maintenance agent is assigned to correct a data-processing defect.

The agent modifies the processing logic and runs the permitted automated

tests.

The tests pass, and the agent reports that the change is complete.

The corrected software processes new records successfully.

However, records previously processed under the defective logic remain

inconsistent.

CCLM evaluates:

Original Change Purpose → Agent Modification → New Processing Results →

Historical Data Condition → Outcome Verification → Residual Obligations

The assessment identifies:

• The software modification was implemented

• The new processing behavior satisfies the relevant functional tests

• Historical data remains affected by the original defect

• The original change objective may not be fully satisfied if correction

• A data remediation obligation must be established and governed

• The agent's task-completion report does not independently authorize

of existing records was part of the required outcome. successful change closure.

The appropriate classification is:

Implementation Verified — Residual Data Integrity Obligation Outstanding.

A qualified closure or transfer may be considered only if the remaining obligation is explicitly identified, legitimately assigned, and does not invalidate the applicable disposition.

Agent Task Completion ≠ Verified Resolution of All Change Consequences.

Critical Distinctions

Implementation Complete ≠ Intended Outcome Verified.

Technical Task Completed ≠ Change Governance Completed.

Deployment Successful ≠ Change Successfully Resolved.

Original Change Purpose ≠ Actual Implementation Result.

Approved Scope ≠ Implemented Scope.

Verification Evidence Available ≠ Verification Coverage Sufficient.

Historical Evidence ≠ Automatically Applicable Evidence.

Partial Success ≠ Full Resolution.

Failed Change Closure ≠ Successful Change Resolution.

Request Rejected ≠ Underlying Problem Resolved.

Request Withdrawn ≠ Underlying Risk Eliminated.

Change Superseded ≠ Original Obligations Erased.

Obligation Transferred ≠ Obligation Resolved.

Residual Obligation Identified ≠ Residual Obligation Satisfied.

Administrative Closure ≠ Operational Finality.

Agent Task Completion ≠ Authorized Change Closure.

Final Record ≠ Immutable Truth About the Resulting Software State.

Canonical Boundary

CCLM governs the evidence-supported evaluation, classification,

disposition, and formal closure of identifiable software change cases.

It establishes the relationship between the original change purpose,

applicable decisions, actual implementation, resulting software state, verified outcome, lifecycle obligations, coordination conditions, residual responsibilities, and legitimate closure authority.

CCLM does not replace specialized software verification, authorize deployment, execute remediation, or eliminate obligations merely through administrative status changes.

Its principal governance output is the:

Software Change Closure Profile

This output preserves the final governance disposition of the change case

and the evidence necessary to distinguish verified success, partial realization, failure, rejection, withdrawal, supersession, deferral, and continuing obligations.

Canonical Closing Statement

A software change is not successfully resolved merely because the modification was completed, the deployment succeeded, or the work item was closed.

A patch may be deployed while the vulnerability remains exploitable.

A defect may be corrected for new transactions while historical data remains inconsistent.

A dependency update may pass local tests while introducing unresolved compatibility problems.

A production change may function correctly while required recovery verification remains incomplete.

An autonomous agent may report successful task completion without establishing the legitimacy or consequences of every action it performed.

And a change may be legitimately rejected or withdrawn without resolving the original condition that caused it to be proposed.

CCLM establishes the methodological structure required to distinguish these

outcomes.

It requires the intended change to remain connected to the actual

implementation and resulting software state.

It preserves the evidence needed to determine whether the original purpose

was achieved.

It reconciles impact, coordination, and lifecycle obligations before

establishing closure eligibility.

It distinguishes successful resolution from partial completion, failed

implementation, and administrative disposition.

It ensures that unresolved obligations retain identifiable responsibility and continuity.

And it preserves the ability to reassess an earlier closure when material new evidence challenges the conclusion on which that closure was based.

The purpose of software change closure is not to make a change case disappear from an active workflow. It is to establish, with evidence, what happened to the change, what outcome was actually achieved, which obligations were satisfied, and what remains governed after the case is closed.

Change Closure Management Module (CCLM) transforms software change closure

from an administrative endpoint into an evidence-supported determination of outcome, responsibility, and lifecycle finality.